

UNIT 2

Image Processing & Feature Extraction

Topics covered:

Edge Detection — Sobel & Canny
Corner Detection — Harris & Shi-Tomasi
Blob Detection — LoG & DoG
Feature Descriptors — SIFT, SURF & ORB
Texture Analysis & Feature Representation

Prerequisite: Unit 1 — image formation, sampling & quantization, color models, basic filtering
Code examples use Python, OpenCV (cv2), NumPy, Matplotlib and scikit-image

Environment Setup

All practicals in this unit assume the following Python environment. Run this once before working through the examples.

```
pip install opencv-contrib-python numpy matplotlib scikit-image

# Quick sanity check
python -c "import cv2; print(cv2.__version__)"
```

Use opencv-contrib-python (not plain opencv-python) — SIFT/SURF live in the contrib modules. SURF is patented and may be disabled in some builds; ORB and SIFT are always free to use.

Common imports used throughout this document:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def show(img, title=''):
    plt.figure(figsize=(6,6))
    cmap = 'gray' if len(img.shape) == 2 else None
    plt.imshow(img, cmap=cmap)
    plt.title(title); plt.axis('off'); plt.show()

img = cv2.imread('sample.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Edge Detection

What exactly is an edge?

- Edges are pixels where image intensity changes sharply — they usually mark object boundaries, surface orientation changes, or texture.
- In a real image, edges usually occur where there is a significant change in:
 - brightness/intensity
 - color
 - texture

Edge detection is almost always the first feature-extraction step in a computer vision pipeline, since edges compress an image into a much smaller, semantically useful representation.

Why Edges Matter

Suppose there is an image of cup but a computer doesn't naturally see "this is a cup."

It sees something more like:

120 121 119 118 120 ...

122 121 120 119 121 ...

125 124 123 122 120 ...

...

Thousands or millions of numbers.

But the **shape of the cup** is represented strongly by its boundaries.

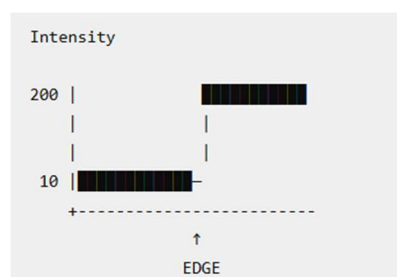
So edge detection converts: **Image** → **important boundaries**

- Boundaries between objects/background carry most of the shape information.
 - Reduces data volume from millions of pixels to a sparse set of edge pixels.
 - Basis for higher-level tasks: segmentation, object detection, shape matching.
-

The basic idea behind Edge Detection

Consider this 1D image:

10 10 10 10 200 200 200 200



Now calculate the change in Intensity (Derivative of Intensity wrt distance)

Intensity of pixels of image: 10 10 10 10 200 200 200

Change in intensity(Derivative): 0 0 0 190 0 0

The derivative is approximately zero where the image doesn't change.

But at the boundary: $200 - 10 = 190 \rightarrow$ derivative becomes large.

- Constant region $\rightarrow$ small derivative
- Edge $\rightarrow$ large derivative

This is the fundamental principle of most classical edge detection.

For an actual image, we don't only move left $\rightarrow$ right.

We can move:

- horizontally
- vertically

Therefore, we calculate derivatives in two directions.

Horizontal derivative

$$G_x = \frac{\partial I}{\partial x}$$

Vertical derivative

$$G_y = \frac{\partial I}{\partial y}$$

Together:

$$\nabla I = \begin{bmatrix} G_x \\ G_y \end{bmatrix}$$

This is called the **gradient**.

The gradient tells us:

1. How strong the change is
2. Which direction the change is happening

At every pixel, gradient magnitude and direction are computed as:

- **Magnitude** (G) = $\sqrt{G_x^2 + G_y^2} \rightarrow$ It tells how strong is the edge at this location?
- Sometimes we use the faster approximation: $G = |G_x| + |G_y|$
- **Gradient Direction** ($\theta = \tan^{-1}\left(\frac{G_y}{G_x}\right) \rightarrow$ This tells us the direction of maximum intensity change.

How do we calculate these derivatives on an image?

- In filtering we used a small matrix (called kernel) and slide it over the image. For edge detection, we use special kernels designed to detect changes.
- The two most famous ones are: **Sobel** and **Prewitt**
- There are also: **Roberts** **Laplacian** and **Canny**

Gradient-Based Edge Detection — Sobel

The Sobel operator approximates the image gradient using two 3×3 convolution kernels —

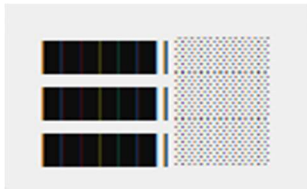
- one for horizontal changes (G_x) and one for vertical changes (G_y).

The kernels are:

Sobel X: $G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$

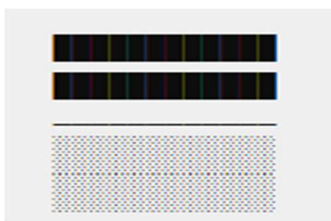
Sobel Y: $G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$

- Sobel X ($\frac{\partial I}{\partial x}$) → compares the left side with the right side (i.e detects **change along x**.)
- So it detects: **Changes in the horizontal direction.**
- Consequently, it is especially strong around **vertical edges**.



Sobel X detects this very well.

- Similarly, Sobel Y ($\frac{\partial I}{\partial y}$) compares the upper region with the lower region (i.e detects **change along y**.)
- Therefore, it detects changes in the vertical direction.
- It is particularly strong around **horizontal edges**.



Sobel Y detects this very well.

At every pixel, gradient magnitude and direction are computed as:

```
Magnitude = sqrt(Gx^2 + Gy^2)
Direction = arctan(Gy / Gx)
```

Larger magnitude $\Rightarrow$ stronger edge. Sobel is fast, simple, and reasonably robust to mild noise because the kernel itself performs a small amount of smoothing.

Practical: Sobel in OpenCV

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = cv2.GaussianBlur(gray, (3,3), 0) # reduce noise first

sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

magnitude = cv2.magnitude(sobel_x, sobel_y)
magnitude = cv2.convertScaleAbs(magnitude) # back to uint8 for display

show(magnitude, 'Sobel Edge Magnitude')
```

Always blur before Sobel/gradient-based methods — raw gradients amplify pixel-level noise heavily.

Original image $\rightarrow$ Gaussian Blur $\rightarrow$ Noise reduction $\rightarrow$ Edge Detection

Because noise can produce false edges.

Gradient-Based Edge Detection — Prewitt Operator

$$\text{Prewitt } X: \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\text{Prewitt } Y: \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Prewitt is almost similar to Sobel just one difference:

- Sobel gives more weight to the center row/column while Prewitt treats them equally
- Sobel is generally preferred because of this additional smoothing effect.

Canny Edge Detector

Canny is the most widely used edge detector because it produces thin, well-localized, single-pixel-wide edges with far fewer false detections than raw gradient thresholding.

It is a **multi-stage edge detection algorithm and works in four stages**:

Noise reduction — Gaussian smoothing to reduce noise.

Gradient computation — Sobel G_x , $G_y \rightarrow$ magnitude & direction.

Non-maximum suppression

It does not search for edges. It takes the already-detected thick edge and makes it 1-pixel thin.

Hysteresis thresholding —

Now Canny classifies pixels based on gradient strength

So here two thresholds are defined

- High threshold
- Low threshold
- Pixels above the high threshold are strong edges,
- Pixels between the two thresholds are (weak edges)
- Hysteresis keeps weak edges only if connected to a strong edge and remove isolated weak edges.
- Pixels below low threshold are discarded.

Practical: Canny in OpenCV

```
edges = cv2.Canny(gray, threshold1=100, threshold2=200)
show(edges, 'Canny Edges')

# Auto-threshold using the median of pixel intensities (common heuristic)
def auto_canny(image, sigma=0.33):
    v = np.median(image)
    lower = int(max(0, (1.0 - sigma) * v))
    upper = int(min(255, (1.0 + sigma) * v))
    return cv2.Canny(image, lower, upper)

auto_edges = auto_canny(gray)
show(auto_edges, 'Auto-thresholded Canny')
```

Choosing the thresholds

- Ratio of high:low threshold is typically 2:1 or 3:1.
- Low threshold too high $\rightarrow$ broken/missing edges.
- High threshold too low $\rightarrow$ noisy, false edges.

Experiment with these values:

```
edges1 = cv2.Canny(gray, 30, 100)
```

```
edges2 = cv2.Canny(gray, 50, 150)
```

```
edges3 = cv2.Canny(gray, 100, 200)
```

```
edges4 = cv2.Canny(gray, 150, 250)
```

Sobel vs Canny

Aspect	Sobel	Canny
Output	Gradient magnitude image (thick edges)	Thin, binary single-pixel edges
Noise handling	Minimal (only kernel smoothing)	Explicit Gaussian blur + hysteresis
Speed	Very fast	Slower (multi-stage)
Use case	Quick gradient/feature maps, further processing	Final edge maps, contour detection, shape analysis
Tunable parameters	Kernel size	Blur sigma, low/high thresholds

Mini-Exercise

- Load any image and compare `cv2.Sobel`, and `cv2.Canny` outputs side-by-side using `plt.subplot`.
- Vary Canny's threshold pair and observe how edge continuity changes.

Corner Detection

- A corner is a point in an image where the intensity changes sharply in two directions simultaneously (unlike an edge, which only varies strongly in one direction).
- **Corner detection tries to find interesting points in an image where the intensity changes strongly in multiple directions.**

These points are also called:

- Corner points / Interest points / Keypoints, depending on the algorithm/context
-

Flat region → little/no intensity change in both directions.

Edge → large change in one direction, small change in another

Corner → large change in multiple directions

```

. . . . .
. . . . .
. . #####
. . #####
. . #####
. . #####
. . #####

```

Harris Corner Detector

Steps to detect corner using Harris Corner Detection Technique:

- 1) Convert Color Image to GrayScale (Because corner detection primarily needs **intensity changes**.)
 - 2) Calculate image gradients using Sobel (**X gradient I_x** and **Y gradient I_y**).
 - a. If I_x and I_y both zero → Flat Region
 - b. I_x = large and I_y = small → Strong change in one direction (**Vertical edge**)
 - c. I_x = small and I_y = large → Strong change in one direction (**Horizontal edge**)
 - d. I_x = large and I_y = large → Strong change in both direction (**Corner**)
-

- Suppose at one pixel: I_x = 50 I_y = 80 → It doesn't automatically mean it's a corner.
- Because One pixel may be noisy so we need to examine the **neighbourhood** around the pixel.
- That's why Harris considers a small window. → For every pixel in the window, we look at: I_x and I_y and combinations of them.

Harris looks at how the sum of squared differences (SSD) changes when a small window is shifted in any direction.

This is captured by the second-moment matrix (structure tensor):

$$M = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$$

Where summation are taken over a small window.

$I_x \rightarrow$ horizontal change

$I_y \rightarrow$ vertical change

$I_x I_y \rightarrow$ helps describe how changes in the two directions are related.

Squaring allows us to measure the **strength of variation**, regardless of whether the intensity is increasing or decreasing.

The matrix M summarizes how much image intensity changes in different directions inside the window.

- 3) Harris then calculates a value called the **Harris response** from the eigenvalues (λ_1, λ_2) of M without explicitly computing them:

$$R = \det(M) - k(\text{trace}(M))^2$$

$$= (\lambda_1 \lambda_2) - k(\lambda_1 + \lambda_2)^2$$

k is a small constant, commonly around: 0.04–0.06

Conceptual explanation of Harris.

- Both λ_1, λ_2 large $\rightarrow R$ large positive $\rightarrow$ corner.
- One large, one small $\rightarrow R$ negative $\rightarrow$ edge.
- Both small $\rightarrow R \approx 0 \rightarrow$ flat region.

Thresholding in Harris

Here we implement some threshold value to achieve: don't want every tiny intensity variation to become a corner.

If Harris Response $>$ Some threshold value $\rightarrow$ Then keep it or discard them

Practical: Harris in OpenCV

```
gray_f = np.float32(gray)
harris = cv2.cornerHarris(gray_f, blockSize=2, ksize=3, k=0.04)
harris = cv2.dilate(harris, None) # enlarge marks for visibility

out = img.copy()
out[harris > 0.01 * harris.max()] = [0, 0, 255] # mark corners in red
show(cv2.cvtColor(out, cv2.COLOR_BGR2RGB), 'Harris Corners')
```

Limitations of Harris:

- Different threshold values can produce different numbers of corners.
- The detector's behaviour depends on the window size. $\rightarrow$ A tiny corner and a large-scale corner may not behave identically.
- Noise creates unwanted intensity variations which can lead to false detection of corners.

Shi-Tomasi (Good Features to Track)

Shi-Tomasi modifies Harris's scoring function $R = \det(M) - k(\text{trace}(M))^2$

Instead of the combined determinant/trace formula, it directly uses the smaller eigenvalue:

$$R = \min(\lambda_1, \lambda_2)$$

A corner is accepted only if this minimum eigenvalue exceeds a threshold — a simpler and often more reliable criterion, which is why this method is the default keypoint selector behind OpenCV's optical-flow tracking pipeline.

Practical: Shi-Tomasi in OpenCV

```
corners = cv2.goodFeaturesToTrack(
    gray, maxCorners=100, qualityLevel=0.01, minDistance=10)
corners = np.int0(corners)

out = img.copy()
for c in corners:
    x, y = c.ravel()
    cv2.circle(out, (x, y), 4, (0, 255, 0), -1)
show(cv2.cvtColor(out, cv2.COLOR_BGR2RGB), 'Shi-Tomasi Corners')
```

Harris vs Shi-Tomasi

Aspect	Harris	Shi-Tomasi
Scoring	$\det(M) - k \cdot \text{trace}(M)^2$	$\min(\lambda_1, \lambda_2)$
Extra parameter	Empirical k (0.04–0.06)	None — more principled
Corner quality control	Threshold on R	<code>maxCorners</code> , <code>qualityLevel</code> , <code>minDistance</code> give direct control
Typical use	Classic corner detection, calibration	Feature points for KLT tracking / optical flow

Blob Detection

A blob is a region that differs in brightness or color from its surroundings and is roughly uniform inside - think of a dark circular spot on a light background.

Edge detection

Sobel/Canny basically ask: **“Where is there a sudden change in intensity?”**

For example:

```
WHITE WHITE WHITE
WHITE WHITE WHITE
-----
BLACK BLACK BLACK
BLACK BLACK BLACK
```

Corner detection

Harris/Shi-Tomasi ask:

“Where does the image intensity change significantly in more than one direction?”

So they detect special points such as:

```

|
|
-----+-----
|
|
```

The + is a corner.

Blob detection

Blob detection asks something different:

“Is there a region here that is significantly different from its surroundings?”

- Edge detection → boundary
- Corner detection → special point
- Blob detection → region/spot

So, the pixels inside the region have some common property that distinguishes them from the surrounding pixels.

That property might be:

- brightness
- darkness
- intensity
- color
- scale
- local contrast

Example:

```

10 10 10 10 10
10 10 [80 80] 10
10 10 [80 80] 10
10 10 10 10 10
10 10 10 10 10

```

Here 80 region is different from the surrounding 10.

Therefore, it can be considered a **blob**.

Laplacian of Gaussian (LoG) to detect Blob

- LoG first smooths the image with a Gaussian of standard deviation σ (Gaussian blur), then
- applies the Laplacian operator (second derivative).

The **Laplacian** is a second-order derivative operator.

- For an image $I(x, y)$

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

Suppose intensity looks like: **Dark** → **Dark** → **Bright** → **Bright**

There is a transition.

First derivative detects the transition.

That's why: **Sobel** → **edge detection**

Second derivative looks at how the rate of change itself changes.

Image → **Gaussian smoothing** → **Laplacian** → **Blob detection**

Blobs appear as strong local maxima/minima of the LoG response.

$$\text{LoG}(x,y) = \nabla^2 [G(x,y,\sigma) * I(x,y)]$$

$$\nabla^2 = \partial^2/\partial x^2 + \partial^2/\partial y^2 \rightarrow (\text{Laplacian})$$

G is the Gaussian and

I is the Image

Changing σ changes the scale at which we look at the image.

For small value of σ the image will be blurred less $\rightarrow$ more minute and variation in intensity will be captured $\rightarrow$ small blob detected.

For large value of σ the image will be blurred more $\rightarrow$ less minute and variation in intensity will be captured $\rightarrow$ large blob detected.

Running LoG at multiple values of σ and searching for extrema across (x, y, σ) gives scale-invariant blob detection — the σ at which the response peaks tells you the blob's characteristic size.

The LoG algorithm — simple version

Step 1 — Take image

Step 2 — Apply Gaussian blur (Use different σ values)

A small Gaussian scale is suitable for the small blob and a larger scale is suitable for the large blob. Therefore, different blob sizes require different scales.

Step 3 — Apply Laplacian for every blurred image.

Gaussian $\sigma=1 \rightarrow$ Laplacian

Gaussian $\sigma=2 \rightarrow$ Laplacian

Gaussian $\sigma=3 \rightarrow$ Laplacian

...

Step 4 — Find pixels/regions where LoG response is strong.

Step 5 — Find local maxima

Step 6 — Mark the blob

LoG is mathematically clean but computationally expensive because it needs a fresh convolution at every scale.

i.e. We need to perform Gaussian + Laplacian operations many times, which is computationally expensive.

*So researchers use an approximation which leads us to **DoG — Difference of Gaussians***

Difference of Gaussians (DoG)

DoG approximates LoG far more cheaply: subtract two Gaussian-blurred versions of the image at nearby scales.

$$\text{DoG}(x, y, \sigma) = G(x, y, k\sigma) * I - G(x, y, \sigma) * I$$

Gaussian with $\sigma_1 \rightarrow G_1$

Gaussian with $\sigma_2 \rightarrow G_2$

$G_2 - G_1 \rightarrow \text{DoG}$

Why subtract two Gaussian images?

Suppose we have:

Image A = lightly blurred

Image B = more heavily blurred

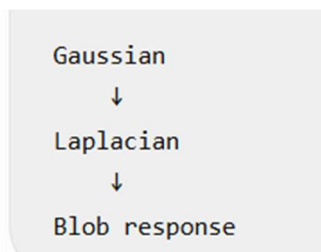
Then: Image B - Image A

highlights structure whose appearance changes significantly between those two scales.

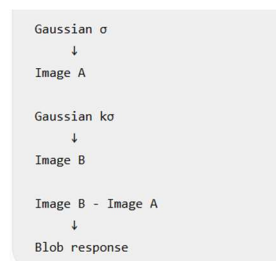
Those structures can correspond to blobs.

DoG detects structures by looking at how the image changes between two nearby scales.

LoG



DoG



Practical: LoG & DoG with scikit-image

```

from skimage.feature import blob_log, blob_dog, blob_doh
from skimage.color import rgb2gray
import matplotlib.pyplot as plt

image_gray = rgb2gray(plt.imread('sample.jpg'))

blobs_log = blob_log(image_gray, max_sigma=30, num_sigma=10, threshold=.1)
blobs_log[:, 2] = blobs_log[:, 2] * np.sqrt(2)  # radius = sigma*sqrt(2)

blobs_dog = blob_dog(image_gray, max_sigma=30, threshold=.1)
  
```

```
blobs_dog[:, 2] = blobs_dog[:, 2] * np.sqrt(2)

fig, ax = plt.subplots()
ax.imshow(image_gray, cmap='gray')
for y, x, r in blobs_log:
    ax.add_patch(plt.Circle((x, y), r, color='red', fill=False, lw=1.5))
plt.title('LoG Blobs'); plt.axis('off'); plt.show()
```

Practical: Simple Blob Detector in OpenCV

```
params = cv2.SimpleBlobDetector_Params()
params.filterByArea = True
params.minArea = 50
params.filterByCircularity = True
params.minCircularity = 0.7

detector = cv2.SimpleBlobDetector_create(params)
keypoints = detector.detect(gray)

out = cv2.drawKeypoints(img, keypoints, None, (0,0,255),
                        cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
show(cv2.cvtColor(out, cv2.COLOR_BGR2RGB), 'Blobs')
```

LoG vs DoG

Feature	LoG	DoG
Full form	Laplacian of Gaussian	Difference of Gaussian
Main idea	Gaussian + Laplacian	Difference between two Gaussian images
Blob detection	Yes	Yes
Computation	More expensive	More efficient
Relationship	Reference method	Approximation of LoG